Continuous exposures and g-computation

Malcolm Barrett

Stanford University

Normal regression estimates associations. But we want ***causal*** estimates: what would happen if ***everyone*** in the study were exposed to x vs if ***no one*** was exposed.

G-Computation/G-Formula

1. Fit a model for `y ~ x + z` where z is all covariates
2. Create a duplicate of your data set for each level of `x`
3. Set the value of x to a single value for each cloned data set (e.g `x = 1` for one, `x = 0` for the other)

G-Computation/G-Formula

1. Make predictions using the model on the cloned data sets
2. Calculate the estimate you want, e.g. `mean(x_1) - mean(x_0)`

Advantages of the parametric G-formula

Often more statistically precise than propensity-based methods

Incredibly flexible

Basis of other important causal models, e.g. causal survival analysis and TMLE

Greek Pantheon data (`greek_data`)

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

+ 10 more rows

1. Fit a model for `y ~ a + l`

```r
greek_model <- lm(y ~ a + l, data = greek_data)
```

2. Create a duplicate of your data set for each level of a

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

2. Create a duplicate of your data set for each level of a

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

The name of a Greek god	A prognostic factor	The treatment, a heart transplant	The outcome, death
Rheia	0	0	0
Kronos	0	0	1
Demeter	0	0	0
Hades	0	0	0
Hestia	0	1	0
Poseidon	0	1	0
Hera	0	1	0
Zeus	0	1	1
Artemis	1	0	1
Apollo	1	0	1

3. Set the value of a to a single value for each cloned data set

The name of a Greek god	A prognostic factor	a	The outcome, death
Rheia	0	**0**	0
Kronos	0	**0**	1
Demeter	0	**0**	0
Hades	0	**0**	0
Hestia	0	**0**	0
Poseidon	0	**0**	0
Hera	0	**0**	0
Zeus	0	**0**	1
Artemis	1	**0**	1
Apollo	1	**0**	1

The name of a Greek god	A prognostic factor	a	The outcome, death
Rheia	0	**1**	0
Kronos	0	**1**	1
Demeter	0	**1**	0
Hades	0	**1**	0
Hestia	0	**1**	0
Poseidon	0	**1**	0
Hera	0	**1**	0
Zeus	0	**1**	1
Artemis	1	**1**	1
Apollo	1	**1**	1

3. Set the value of a to a single value for each cloned data set

```r
#  set all participants to have a = 0
untreated_data <- greek_data |>
  mutate(a = 0)

#  set all participants to have a = 1
treated_data <- greek_data |>
  mutate(a = 1)
```

4. Make predictions using the model on the cloned data sets

```r
#  predict under the data where everyone is untreated
predicted_untreated <- greek_model |>
  augment(newdata = untreated_data) |>
  select(untreated = .fitted)

#  predict under the data where everyone is treated
predicted_treated <- greek_model |>
  augment(newdata = treated_data) |>
  select(treated = .fitted)

predictions <- bind_cols(
  predicted_untreated,
  predicted_treated
)
```

5. Calculate the estimate you want

```r
predictions |>
  summarise(
    mean_treated = mean(treated),
    mean_untreated = mean(untreated),
    difference = mean_treated - mean_untreated
  )
```

```
# A tibble: 1 × 3
  mean_treated mean_untreated difference
         <dbl>          <dbl>      <dbl>
1          0.5            0.5          0
```

`avg_comparisons()` does all five steps

```r
library(marginaleffects)

avg_comparisons(greek_model, variables = "a")
```

```
# A tibble: 1 × 4
  estimate std.error conf.low conf.high
     <dbl>     <dbl>    <dbl>     <dbl>
1        0      0.24    -0.47      0.47
```

The standard error comes from the **delta method**, which approximates the variance of a transformation of the estimated coefficients

Targeting estimands with `newdata`

`newdata` changes whose covariates we average over

```r
# ATT: average over the treated
avg_comparisons(
  greek_model,
  variables = "a",
  newdata = filter(greek_data, a == 1)
)

# ATC: average over the untreated
avg_comparisons(
  greek_model,
  variables = "a",
  newdata = filter(greek_data, a == 0)
)
```

Same model, three populations

```
# A tibble: 3 × 4
  estimand estimate conf.low conf.high
  <chr>       <dbl>    <dbl>     <dbl>
1 ATE             0    -0.47      0.47
2 ATT             0    -0.47      0.47
3 ATC             0    -0.47      0.47
```

Without an exposure-covariate interaction, the three coincide: the model assumes a constant effect across covariate values

Tilting toward a target population

The ATO and ATM populations are not subsets of the rows, so weight the average by the tilting function instead

```r
library(propensity)

greek_ps <- glm(a ~ 1, data = greek_data, family = binomial()) |>
  predict(type = "response")

avg_comparisons(
  greek_model,
  variables = "a",
  wts = ps_tilt(greek_ps, "ato")
)
```

```
  Estimate Std. Error         z Pr(>|z|)    S  2.5 % 97.5 %
 -1.31e-17       0.24 -5.44e-17        1 -0.0 -0.471  0.471

Term: a
Type: response
Comparison: 1 - 0
```

Continuous exposures

We recommend g-computation over propensity scores for continuous exposures because of stability issues

Does change in smoking intensity (`smkintensity82_71`) affect weight gain among lighter smokers?

```
nhefs_light_smokers <- nhefs_complete |>
  filter(smokeintensity <= 25)
```

1. Fit a model for `y ~ x + z`

```r
nhefs_gcomp_model <- lm(
  wt82_71 ~ splines::ns(smkintensity82_71, df = 3) +
    sex + race + age + I(age^2) +
    education + smokeintensity + I(smokeintensity^2) +
    smokeyrs + I(smokeyrs^2) + exercise + active +
    wt71 + I(wt71^2),
  data = nhefs_light_smokers
)
```

G-computation still uses a single outcome model, and a continuous exposure lets us fit it flexibly with a natural spline. The remaining steps are identical, except that the levels of step 2 become the exposure values we choose to compare

2 and 3. Set `smkintensity82_71` in each cloned data set

```r
#  set all participants to have no change in intensity
unchanged_data <- nhefs_light_smokers |>
  mutate(smkintensity82_71 = 0)

#  set all participants to smoke 10 fewer cigarettes
reduced_data <- nhefs_light_smokers |>
  mutate(smkintensity82_71 = -10)
```

4. Make predictions using the model on the cloned data sets

```r
#  predict under the data where intensity is unchanged
predicted_unchanged <- nhefs_gcomp_model |>
  augment(newdata = unchanged_data) |>
  select(unchanged = .fitted)

#  predict under the data where everyone reduces by 10
predicted_reduced <- nhefs_gcomp_model |>
  augment(newdata = reduced_data) |>
  select(reduced = .fitted)

nhefs_predictions <- bind_cols(
  predicted_unchanged,
  predicted_reduced
)
```

5. Calculate the estimate you want

```r
nhefs_predictions |>
  summarise(
    mean_unchanged = mean(unchanged),
    mean_reduced = mean(reduced),
    difference = mean_reduced - mean_unchanged
  )
```

```
# A tibble: 1 × 3
  mean_unchanged mean_reduced difference
           <dbl>        <dbl>      <dbl>
1           1.77         3.21       1.44
```

`avg_comparisons()` with chosen exposure values

```r
avg_comparisons(
  nhefs_gcomp_model,
  variables = list(smkintensity82_71 = c(-10, 0))
)
```

```
# A tibble: 1 × 4
  estimate std.error conf.low conf.high
     <dbl>     <dbl>    <dbl>     <dbl>
1    -1.44      0.39    -2.21     -0.67
```

`avg_comparisons()` contrasts the higher value against the lower one, so the estimate is the mean at 0 minus the mean at -10. Reversing it gives the answer from step 5: reducing intensity by 10 cigarettes a day increases weight gain by 1.44 kg

Predict across the exposure range

```
nhefs_dose_response <- avg_predictions(
  nhefs_gcomp_model,
  variables = list(smkintensity82_71 = seq(-25, 25, by = 5))
)
```

With a continuous exposure, we don't have to settle for a single contrast: we can ask what the average weight gain would be at every value of the exposure

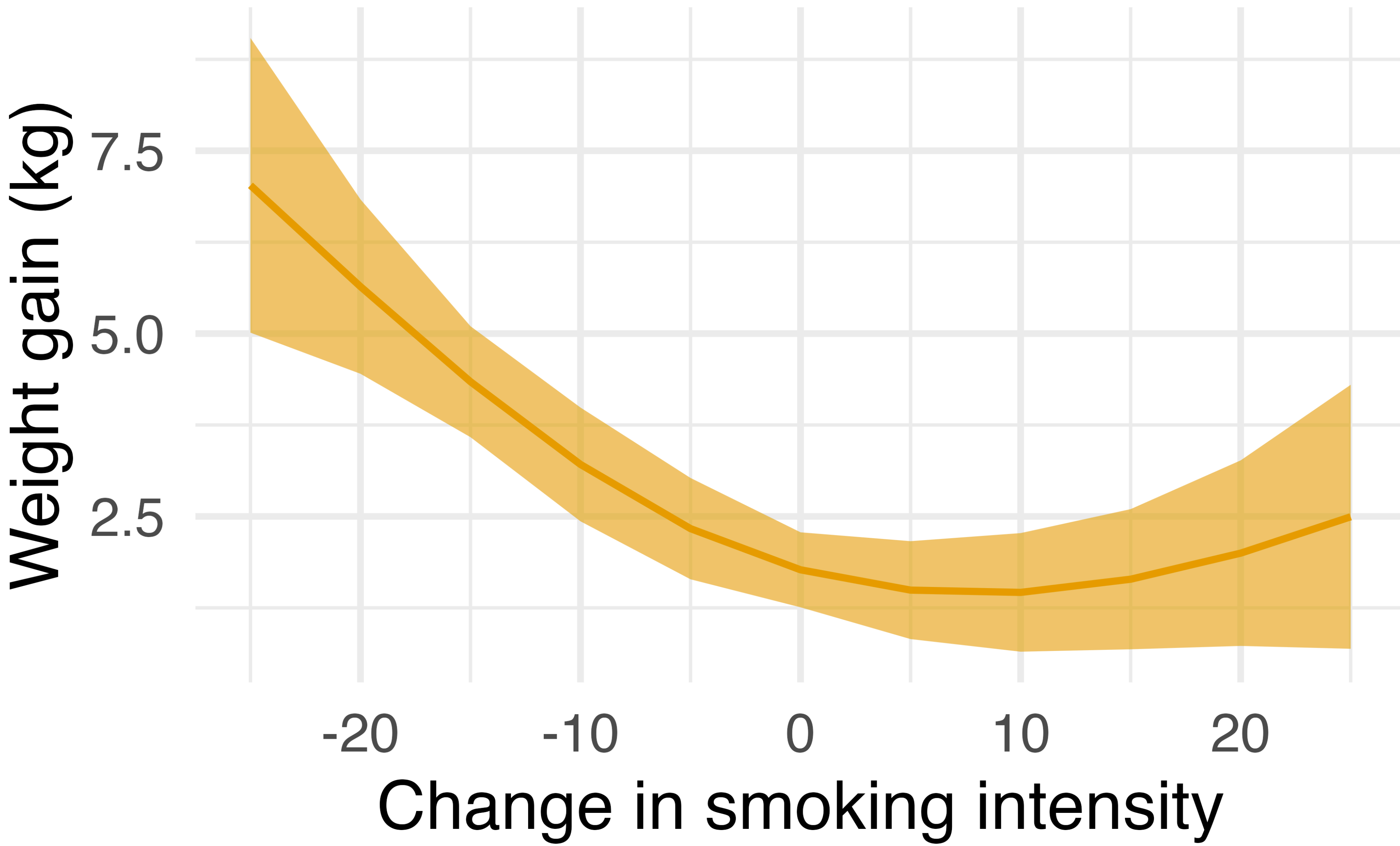
Predict across the exposure range

```
nhefs_dose_response
```

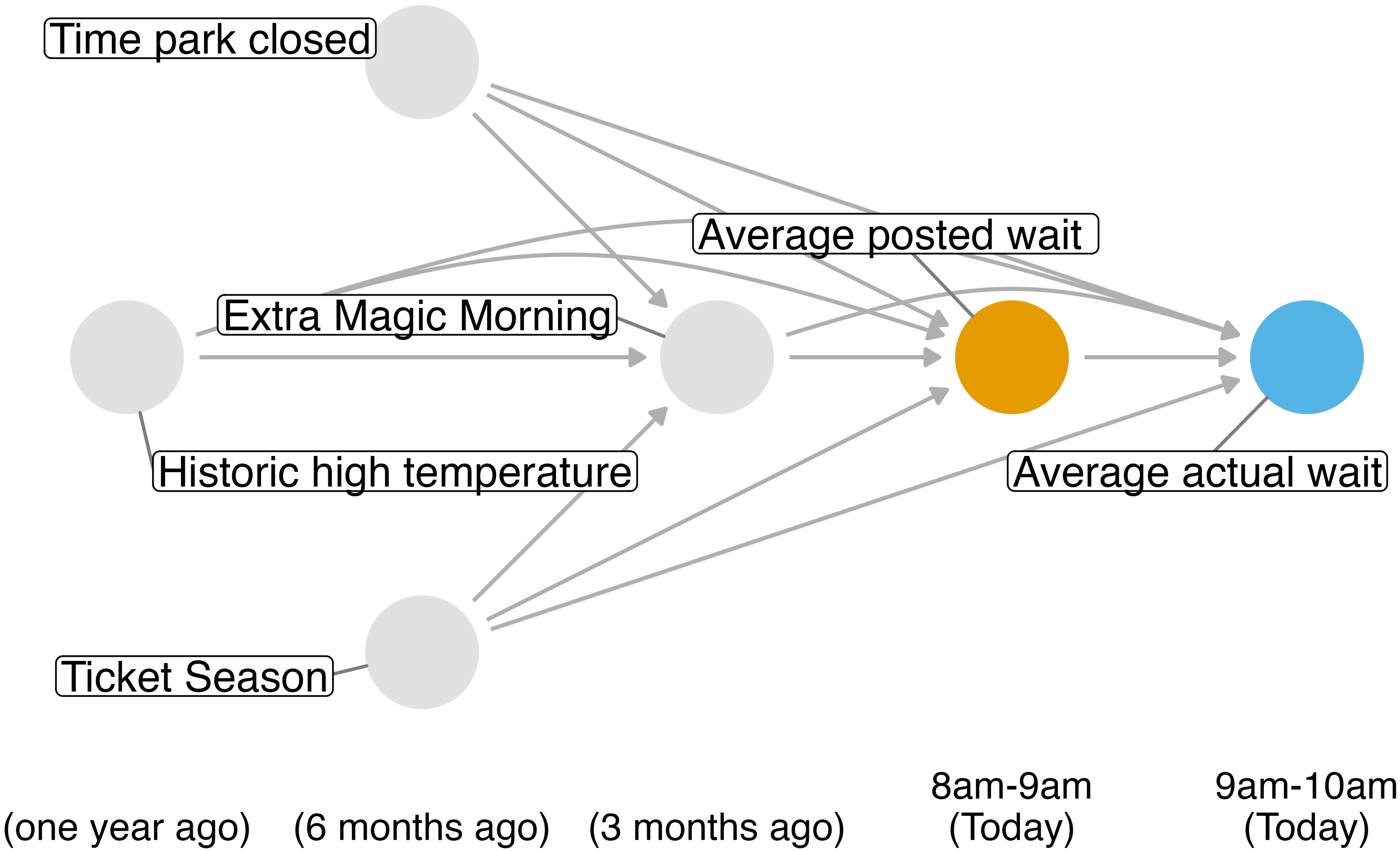
```
 smkintensity82_71 Estimate Std. Error     z Pr(>|z|)    S 2.5 % 97.5 %
               -25     7.03      1.028  6.84  < 0.001 36.8 5.013   9.04
               -20     5.64      0.608  9.28  < 0.001 65.7 4.451   6.83
               -15     4.34      0.387 11.22  < 0.001 94.7 3.584   5.10
               -10     3.21      0.397  8.09  < 0.001 50.6 2.433   3.99
                -5     2.33      0.353  6.60  < 0.001 34.5 1.640   3.03
                 0     1.77      0.261  6.77  < 0.001 36.2 1.257   2.28
                 5     1.49      0.341  4.37  < 0.001 16.3 0.823   2.16
                10     1.46      0.414  3.53  < 0.001 11.2 0.650   2.27
                15     1.64      0.489  3.36  < 0.001 10.3 0.683   2.60
                20     2.00      0.647  3.09  0.00202  8.9 0.729   3.27
                25     2.49      0.921  2.71  0.00674  7.2 0.690   4.30

Type: response
```

The causal dose-response function



Do *posted* wait times at 8 am affect *actual* wait times at 9 am?



Your Turn

Work through Your Turns 1-4 in `11-continuous-g-computation-exercises.qmd`

10:00